

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 4 : SLO – 2 | Topological Sorting in a DAG

■ I. Fill in the Blanks

- Topological sorting is only possible for DIRECTED ACYCLIC GRAPHS (DAGs).
- In topological sorting, the PREDECESSORS (dependencies) of each vertex appears before the vertex itself.
- The time complexity of topological sorting using DFS is $O(V + E)$.
- In topological sort, if a graph contains a cycle, the sort is IMPOSSIBLE (not possible).
- DFS-BASED method and KAHN'S ALGORITHM (BFS/in-degree based) are two standard methods for topological sorting.

■ II. True or False

- Topological sorting applies to both directed and undirected graphs — FALSE (directed acyclic graphs only).
- A topological sort can have multiple valid orderings for the same graph — TRUE.
- In a topological sort, all outgoing edges from a vertex are considered before visiting it — FALSE
- (all INCOMING dependencies must be resolved first).
- Topological sorting is useful for scheduling tasks based on dependencies — TRUE.

■ III. Short Answer Questions

- Topological sorting: a linear ordering of the vertices of a DAG such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering.
- Applications: course prerequisite scheduling; build systems / task dependency resolution (e.g. Makefile), also package installation ordering.
- Cyclic graph: NO — a cycle means each vertex in the cycle depends on another, so no vertex can come first;
- a topological order exists if and only if the graph is acyclic.

■ IV. Complete the Missing Parts (DFS-based Topological Sort)

1. Mark all vertices as UNVISITED
2. For each unvisited vertex: a. Call the TOPOLOGICAL SORT (DFS utility) function
3. In the recursive function:
 - a. Mark the vertex as VISITED
 - b. Recursively call for all its unvisited ADJACENT VERTICES (neighbours)
 - c. Push the vertex onto the STACK after visiting all its neighbours
 - (Finally, pop the stack to get the topological order.)

■ V. Solve and Write — Given DAG (edges 7->5, 7->6, 5->2, 5->4, 6->4, 6->3, 2->1, 3->1, 1->0)

- Two valid topological orders:
- 7, 5, 6, 4, 2, 3, 1, 0

- 7, 6, 5, 2, 3, 4, 1, 0
- Vertex that must always appear before vertex 0: VERTEX 1 (its only direct predecessor; $1 \rightarrow 0$).

■ VI. Spot the Error

- Incorrect: 'a vertex is placed before all its incoming edges.'
- Correct: In a topological sort, a vertex is placed AFTER all vertices that have edges coming INTO it
- (i.e. all its predecessors/dependencies appear before the vertex); equivalently, for every edge
- $u \rightarrow v$, u appears before v .

■ VII. Challenge Question

- A straight chain $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n$ has exactly 1 valid topological order: 1, 2, 3, ..., n .
- Every vertex's position is forced by its single dependency chain.